SECURE DIGITAL VAULT : A SECURE FILE STORAGE AND MANAGEMENT SYSTEM

Mini Project submitted in partial fulfillment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING - CYBER SECURITY

By

Gadda Akhil 23R11A6212

Kondeti Ashvitha 23R11A6224

Veerareddy Rishikesh Reddy 23R11A6246

Under the esteemed guidance

Of

N. Radhika

Assistant Professor

Department of CSE-Cyber Security

Geethanjali College of Engineering and Technology

(UGC Autonomous)

(Affiliated to J.N.T.U.H, Approved by AICTE, New Delhi, NAAC A+)

Cheeryal (V), Keesara (M), Medchal Dist.-501 301.

August-2026

Geethanjali College of Engineering and Technology

(UGC Autonomous)
(Affiliated to JNTUH, Approved by AICTE, New Delhi, Accredited by NBA)
Cheeryal (V), Keesara(M), Medchal Dist.-501 301.

DEPARTMENT OF CSE - CYBER SECURITY



CERTIFICATE

This is to certify that the B.Tech Mini Project report entitled " **SECURE DIGITAL VAULT : A SECURE FILE STORAGE AND MANAGEMENT SYSTEM** " is a bonafide work done by Gadda Akhil **23R11A6212**, Kondeti Ashvitha **23R11A6224**, Veerareddy Rishikesh Reddy **23R11A6246** in partial fulfillment of the requirement of the award for the degree of Bachelor of Technology in "**CSE - Cyber Security**" from Jawaharlal Nehru Technological University, Hyderabad during the year 2026-2027.

Internal Guide

N. Radhika

Assistant Professor

HOD CSE-CS

Dr. K. Krishna Jyothi

Associate professor

External Examiner

Geethanjali College of Engineering & Technology

(UGC Autonomous)
(Affiliated to JNTUH Approved by AICTE, New Delhi, Accredited by NBA)
Cheeryal (V), Keesara(M), Medchal Dist.-501 301.

DEPARTMENT OF CSE - CYBER SECURITY

DECLARATION BY THE CANDIDATE

We, Gadda Akhil, Kondeti Ashvitha, Veerareddy Rishikesh Reddy bearing Roll Nos. **23R11A6212, 23R11A6224, 23R11A6246**, hereby declare that the mini-project report entitled "**SECURE DIGITAL VAULT : A SECURE FILE STORAGE AND MANAGEMENT SYSTEM**" is done under the guidance of **N. Radhika**, Assistant Professor, Department of Computer Science and Engineering - Cyber Security, Geethanjali College of Engineering and Technology, is submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in CSE-Cyber Security

This is a record of bonafide work carried out by us in Geethanjali College of Engineering and Technology and the results embodied in this project have not been reproduced or copied from any source. The results embodied in this project report have not been submitted to any other University or Institute for the award of any other degree or diploma.

Gadda Akhil 23R11A6212,
Kondeti Ashvitha 23R11A6224,
Veerareddy Rishikesh Reddy 23R11A6246.

ACKNOWLEDGEMENT

We would like to extend our sincere gratitude to our distinguished Chairman**, Sri. G. R. Ravinder Reddy**, whose steadfast support has been indispensable to the successful completion of our Mini Project. We also express our heartfelt gratitude to our esteemed Vice Chairman, **Sri. K. Harish Chandra Reddy**, for his valuable support and encouragement throughout this journey. Our sincere appreciation goes to our respected Director, **Dr. S. Udaya Kumar**, GCET, whose guidance and insights have been instrumental in shaping our project. We are immensely grateful to our esteemed Principal, **Dr. K. Sagar**, GCET, for his unwavering support and direction during this entire process. Additionally, we extend our gratitude to **Dr. V. Madhusudan Rao**, Professor and Dean, School of Computer Science and Informatics, for his continuous support and motivation.

We are immensely grateful to **Dr. K. Krishna Jyothi,** Association professor, Head of Department, Department of CSE - CS, for her expertise, encouragement, and invaluable support, which have been instrumental in shaping our project's direction and outcomes. We express our heartfelt gratitude to our Project Guide, **N. Radhika**, Assistant Professor, Department of CSE - Cyber Security for her insightful suggestions, continuous support, and invaluable mentorship, which have greatly contributed to the successful completion of our project.

Lastly, we would like to express our heartfelt gratitude to our friends and family for their unwavering support, understanding, and encouragement throughout this endeavor. Their patience and belief in us have been a constant source of motivation, and we are immensely grateful for their presence in our lives.

Gadda Akhil 23R11A6212,
Kondeti Ashvitha 23R11A6224,
Veerareddy Rishikesh Reddy 23R11A6246.

ABSTRACT

The Secure Digital Vault is a web-based application designed to securely store and manage sensitive digital files while ensuring confidentiality, integrity, and controlled access. It incorporates essential cybersecurity techniques to protect user data from unauthorized access and potential threats. User authentication is implemented using strong credentials, with passwords securely stored through hashing algorithms such as bcrypt or Argon2. When a user uploads a file, it is encrypted using AES-256 encryption before storage, ensuring that the data remains protected. To maintain data integrity, a unique SHA-256 hash is generated for each file, and during retrieval, the system verifies the file by comparing stored and newly generated hashes to detect any unauthorized modifications. The system also includes activity monitoring to identify suspicious actions such as repeated failed login attempts, unusual access patterns, or unauthorized file modifications, and sends immediate notifications to users for quick response. Additionally, the database is secured by restricting direct public access and allowing only backend-mediated interactions, supported by strong authentication mechanisms and access control policies. Overall, the Secure Digital Vault provides a reliable and efficient solution for secure digital file storage by integrating encryption, integrity verification, secure authentication, database protection, and real-time alert systems, ensuring a high level of data security and user trust.

Keywords: *Secure Digital Vault, Cybersecurity, AES-256 Encryption, SHA-256 Hashing, JWT Authentication, bcrypt, Secure File Storage, Data Integrity.*

LIST OF FIGURES

Figure No.	Figure	Page No.
4.1	Use Case Diagram	21
4.2	Class Diagram	22
4.3	Sequence Diagram	23
4.4	Activity Diagram	23
6.1	Password Hashing Algorithm Comparison	29
7.1	Dashboard – Live Vault Overview	30
7.2	My Vault – Encrypted File Explorer	31
7.3	Upload – Encryption Pipeline	31
7.4	Activity Log – Full Audit Trail	32
7.5	Alerts – Real-Time Security Notifications	32
7.6	Access Control – Roles & Database Policies	33
7.7	Reports – Security Summary & Export	33
11.1	Plagiarism Report	37

TABLE OF CONTENTS

Abstract v
List of Figures vi
1. Introduction 9
1.1 About the Project 9
1.2 Objectives 10
2. System Analysis 11
2.1 Existing System 12
2.2 Proposed System 13
2.3 Feasibility Study 14
2.4 Scope of the Project 15
2.5 Modules Description 16
2.6 System Configuration 17
3. Literature Overview 18
4. System Design 19
4.1 Use Case Diagram 21
4.2 Class Diagram 22
4.3 Sequence Diagrams 23
5. Implementation 24
5.1 Secure Authentication (bcrypt/Argon2 + JWT) 24
5.2 AES-256 File Encryption Pipeline 25
5.3 SHA-256 Integrity Verification 25
5.4 Building the React.js Vault Dashboard 26
5.5 Activity Monitoring and Alert Generation 26
5.6 Result Summary 27
5.7 Summary of Implementation Flow 27
6. Testing 28
6.1 Password Hashing Algorithm Comparison 29
7. Output Screens 30
7.1 Dashboard - Live Vault Overview 31
7.2 My Vault - Encrypted File Explorer 31
7.3 Upload - Encryption Pipeline 31
7.4 Activity Log - Full Audit Trail 32
7.5 Alerts - Real-Time Security Notifications 32
7.6 Access Control - Roles & Database Policies 33
7.7 Reports - Security Summary & Export 33

8. Conclusion 34
9. Bibliography 35
9.1 Books and Research Papers 34
9.2 Web and Online Resources 34
9.3 Standards and Guidelines 34
10. Appendices 36
11. Plagiarism Report 37

1. INTRODUCTION

1.1 About the Project

The Secure Digital Vault is a web-based cybersecurity application built to give individuals and small organizations a trustworthy place to store sensitive digital files. Unlike a conventional file-sharing or cloud-storage tool, the vault treats confidentiality, integrity, and controlled access as first-class requirements rather than optional add-ons. Every file that enters the system is encrypted before it ever touches persistent storage, every stored file carries a cryptographic fingerprint that lets the system detect tampering, and every authentication attempt is checked against securely hashed credentials rather than plaintext passwords.

The platform is built using a React.js front end for the interactive vault dashboard and a Node.js/Express backend that mediates all access to the database and file storage. Sensitive operations - password verification, AES-256 encryption and decryption, SHA-256 hash generation and comparison, and JWT token issuance - are all performed on the backend, so the browser never has direct access to raw cryptographic material beyond what a session legitimately requires. The system also keeps a continuous activity log of logins, uploads, downloads, and permission changes, and an alerting module watches this log for patterns that suggest brute-force attempts, unusual access times, or unexpected file modification.

By combining strong authentication, symmetric-key file encryption, hash-based integrity verification, real-time activity monitoring, and strict database access control into a single, approachable dashboard, the Secure Digital Vault demonstrates how modern cybersecurity primitives can be assembled into a practical, everyday tool for protecting sensitive digital assets - without requiring the end user to understand the cryptography working behind the scenes.

1.2 Objectives

- To design and implement a secure user authentication system using salted password hashing (bcrypt / Argon2) and stateless session management via JSON Web Tokens (JWT).
- To encrypt every uploaded file using AES-256 before it is written to storage, so that raw file content is never persisted or transmitted in plaintext.

- To generate a SHA-256 integrity hash for each file at upload time and re-verify it at every retrieval, so that any unauthorized modification or corruption is detected immediately.
- To continuously monitor user and system activity - logins, uploads, downloads, permission changes - and flag suspicious patterns such as repeated failed logins or abnormal access volume.
- To deliver real-time notifications to users and administrators the moment a high-severity security event is detected, enabling a fast response.
- To restrict all database interaction to backend-mediated, authenticated API calls, eliminating any direct public access path to stored data.
- To provide role-based access control so that file owners can precisely define who may view, edit, or share a given file or folder.
- To present all of the above through a single, intuitive dashboard covering vault browsing, upload, activity auditing, alerts, access management, and compliance reporting.

Taken together, these objectives are meant to demonstrate that a small, well-designed system can deliver enterprise-grade data protection - encryption at rest, verifiable integrity, strong authentication, and continuous monitoring - without the operational overhead of a full commercial data-loss-prevention suite, making it suitable for students, freelancers, and small teams who need genuine security guarantees for their most sensitive files.

2. SYSTEM ANALYSIS

2.1 Existing System

Most general-purpose cloud storage and file-sharing tools available today (consumer drives, shared network folders, plain FTP servers) are optimized for convenience and collaboration rather than for provable security. Files are frequently stored as-is on disk, protected only by the platform's own account login and coarse folder permissions. Encryption, where it exists at all, is usually applied at the infrastructure level by the provider and is invisible and unverifiable to the end user; there is rarely any client-facing mechanism to prove that a downloaded file is bit-for-bit identical to what was originally uploaded. Password security is also inconsistent - many small, self-hosted, or legacy systems still store passwords using weak or unsalted hashes, or rely on the transport layer (HTTPS) as their only real protection.

2.1.1 Disadvantages of Existing System

- No client-verifiable integrity guarantee: users have no easy way to confirm that a retrieved file has not been altered, corrupted, or tampered with while at rest.
- Weak or absent password hashing: some legacy systems use fast, unsalted hash functions (e.g. plain MD5/SHA-1) that are vulnerable to rainbow-table and brute-force attacks.
- Lack of real-time monitoring: repeated failed logins, unusual download volumes, or off-hours access frequently go unnoticed until after a breach has already occurred.
- Direct or loosely-guarded database access: some deployments expose database endpoints directly to application servers without a strict authenticated mediation layer, widening the attack surface.
- Coarse access control: many systems offer only binary "shared / not shared" permissions rather than granular owner/editor/viewer roles.
- No unified audit trail: activity across uploads, downloads, and permission changes is often scattered across separate logs, making incident investigation slow and error-prone.

2.2 Proposed System

The proposed system, Secure Digital Vault, addresses each of these gaps directly. Every file is encrypted with AES-256 immediately on upload, using keys that are never exposed to the browser.

A SHA-256 hash is computed at upload time and stored alongside the encrypted blob; on every download, the system re-computes the hash of the decrypted content and compares it against the stored value, immediately flagging any mismatch as a possible integrity violation. Passwords are never stored or compared in plaintext - they are hashed using bcrypt or Argon2, both of which are deliberately slow and salted to resist brute-force and rainbow-table attacks. Authenticated sessions are managed using short-lived JWT tokens rather than persistent server-side sessions, reducing the attack surface for session hijacking.

An Activity Monitoring module logs every significant event - login success/failure, upload, download, permission change - and an Alert Manager continuously evaluates this stream against simple rule-based thresholds (e.g. three or more failed logins within five minutes, an unusually large number of downloads in a short window, or a hash mismatch on retrieval) to raise Critical, High, Medium, or Low severity alerts, which are pushed to the dashboard as real-time toast notifications. Finally, the database itself is never exposed to the public internet or to the front end directly; all reads and writes pass through an authenticated backend API layer that enforces role-based access control and uses parameterized queries to prevent injection attacks.

2.2.1 Advantages of Proposed System

- End-to-end confidentiality: files are encrypted with AES-256 before storage and only decrypted on demand for an authorized, authenticated request.
- Provable integrity: SHA-256 hashing lets the system detect - rather than merely hope against - any tampering or corruption of stored files.
- Strong credential protection: bcrypt/Argon2 hashing with per-user salts makes offline password-cracking computationally expensive.
- Stateless, revocable sessions: JWT-based authentication supports short expiry windows and easy session invalidation.
- Real-time situational awareness: the Activity Monitor and Alert Manager surface suspicious behaviour within seconds rather than after the fact.
- Reduced attack surface: the database is never directly reachable, and every query is parameterized and access-controlled.

- Granular access control: Owner, Editor, Viewer, and Auditor roles let users share precisely as much access as intended, no more.

2.2.2 Modules

Authentication Module

Handles user registration and login. Passwords are hashed with bcrypt or Argon2 before being stored, and every successful login issues a signed JWT that the client must present on subsequent requests. Failed attempts are logged and rate-limited.

File Encryption Module

Encrypts every uploaded file using AES-256 in the backend before it is written to storage, and decrypts it only when a verified, authorized request is made. Encryption keys are managed server-side and are never transmitted to or stored in the browser.

Integrity Verification Module

Computes a SHA-256 hash of each file at upload time and stores it as metadata. On every retrieval, the module recomputes the hash of the decrypted content and compares it against the stored value, flagging a mismatch as a potential tampering event.

Activity Monitoring Module

Continuously logs authentication events, file operations, and permission changes, and evaluates them against threshold-based rules to detect brute-force attempts, unusual access patterns, and other anomalies.

Alerting and Notification Module

Consumes the output of the Activity Monitoring module and raises severity-classified alerts (Critical/High/Medium/Low) as real-time toast notifications on the dashboard, ensuring immediate visibility of high-risk events.

Access Control Module

Implements role-based permissions (Owner, Editor, Viewer, Auditor) at the file and folder level, and enforces them on every backend request so that a user can never act beyond their assigned role.

Database Security Module

Ensures the database is reachable only through the authenticated backend API, using parameterized queries and encrypted connection credentials stored in environment variables rather than in source code.

Reporting Module

Generates downloadable security summary reports covering file counts, integrity check results, alert history, and a compliance checklist, suitable for documentation or internal review.

2.3 Feasibility Study

The feasibility of the Secure Digital Vault was assessed across technical, economic, operational, and scheduling dimensions to confirm that the system can realistically be built, deployed, and maintained by a small student team within an academic timeframe.

2.3.1 Details

Technically, the project relies entirely on mature, well-documented, open-source technologies: Node.js and Express for the backend API, React.js for the dashboard, native and well-audited cryptography libraries for AES-256, SHA-256, and bcrypt/Argon2, and a standard relational or document database for encrypted metadata storage. None of these components require specialized hardware, and the entire system can run comfortably on a single modest server or even a developer laptop for demonstration purposes. The modular design - separate Authentication, Encryption, Integrity, Monitoring, and Access Control modules - means each piece can be implemented, tested, and upgraded independently.

2.3.2 Impact on Environment

As a purely software-based system, the Secure Digital Vault has a minimal direct environmental footprint. It does not require specialized hardware and can run on existing computing

infrastructure, meaning its environmental impact is limited to ordinary electricity consumption for computation and storage, which is comparatively small next to the resources saved by replacing paper-based document handling and manual record-keeping with a digital, auditable alternative.

2.3.3 Safety

The system's safety considerations centre on protecting data and access rather than physical hazards. Because every file is encrypted before storage and the database is never directly exposed, the risk of a single point of failure exposing raw sensitive data is greatly reduced. Sensitive configuration values such as database credentials and encryption key material are managed through environment variables rather than hard-coded values, and the backend runs in an isolated runtime environment to reduce the chance of dependency conflicts or instability affecting security guarantees.

2.3.4 Ethics

The Secure Digital Vault is designed to strengthen, not undermine, personal and organizational data privacy. It stores only the files and metadata the user explicitly uploads, never inspects file content beyond what encryption and hashing require, and gives users full visibility into who has accessed their files through the activity log. Transparency around hashing algorithms, encryption standards, and access policies is maintained throughout the dashboard so that users can trust and verify the protections in place, and the system is explicitly intended for lawful protection of one's own or authorized data rather than any form of unauthorized surveillance or data exfiltration.

2.3.5 Cost

Because the project is built entirely on open-source frameworks and libraries (React.js, Node.js/Express, standard cryptography libraries, and a free-tier-capable database), licensing costs are essentially zero. The primary costs are modest hosting or server costs for production deployment and, optionally, a managed database service if the project is scaled beyond a single small server - both of which remain affordable for academic, personal, or small-business use.

2.3.6 Type

The Secure Digital Vault addresses several classes of digital-security risk: credential-based attacks (brute-force and credential-stuffing login attempts), data-at-rest exposure (unencrypted or weakly

protected file storage), data-integrity attacks (silent tampering or corruption of stored files), and unauthorized-access risk (users or processes attempting to reach files or database records beyond their granted role). By addressing authentication, encryption, integrity, and access control together, the system provides layered protection against all four categories rather than any single point solution.

2.4 Scope of the Project

The Secure Digital Vault is scoped to provide secure upload, storage, retrieval, and sharing of digital files for individual users and small teams, backed by strong authentication, AES-256 encryption, SHA-256 integrity verification, activity logging, and real-time alerting. It supports role-based sharing of individual files and folders, and produces exportable security summary reports for documentation and compliance purposes. The scope is deliberately focused on demonstrating sound cryptographic and access-control practice rather than on large-scale enterprise features such as multi-region replication or hardware security modules, though the modular architecture leaves room for such extensions in future work.

2.5 Modules Description

The Authentication Module manages registration, login, and JWT issuance, verifying credentials against securely hashed passwords. The File Encryption Module encrypts every file with AES-256 immediately on upload and decrypts it only for authorized, verified requests. The Integrity Verification Module computes and later re-verifies a SHA-256 hash for every file, guaranteeing that any silent modification is detected. The Activity Monitoring Module records every security-relevant event and evaluates it for anomalies, feeding the Alerting Module, which raises severity-classified, real-time notifications. The Access Control Module enforces Owner/Editor/Viewer/Auditor roles on every request, and the Database Security Module ensures that all of this happens through a single, authenticated backend gateway rather than any direct data-layer exposure. Finally, the Reporting Module lets users export a summary of the vault's security posture at any time.

2.6 System Configuration

2.6.1 Software Configuration

- Node.js 20 LTS - backend runtime for the authentication, encryption, and API layer.
- Express.js - REST API framework mediating all database and file-storage access.
- React.js 18 - frontend framework for the interactive vault dashboard.
- bcrypt / Argon2 (via native libraries) - password hashing.
- Node crypto module (AES-256-CBC/GCM, SHA-256) - file encryption and integrity hashing.
- jsonwebtoken - JWT issuance and verification.
- MongoDB / PostgreSQL - encrypted metadata and audit-log storage.
- Visual Studio Code - development environment.

2.6.2 Hardware Configuration

- Operating System: Ubuntu 22.04 LTS, Windows 10/11, or macOS
- Processor: Dual-core 2.0 GHz or higher
- RAM: 8 GB minimum (16 GB recommended for development)
- Storage: 20 GB free space for application, dependencies, and encrypted file storage
- Browser: Google Chrome / Mozilla Firefox for dashboard access
- Version Control: Git

3. LITERATURE OVERVIEW

Secure file storage sits at the intersection of applied cryptography and systems design, and a substantial body of prior work informs the approach taken in the Secure Digital Vault. The Advanced Encryption Standard (AES), standardized by NIST in FIPS 197, remains the benchmark symmetric cipher for encrypting data at rest, and AES-256 in particular is widely recommended wherever long-term confidentiality is required. Cryptographic hash functions from the SHA-2 family, especially SHA-256, are the standard mechanism for verifying data integrity, since even a single-bit change in a file produces a completely different hash, making tampering or corruption immediately detectable.

On the authentication side, research and industry guidance (including OWASP's password-storage recommendations) consistently discourage fast, general-purpose hash functions such as plain SHA-256 or MD5 for password storage, since their speed makes brute-force and rainbow-table attacks practical at scale. Purpose-built, deliberately slow, salted algorithms such as bcrypt and Argon2 (winner of the Password Hashing Competition) are recommended instead, since they can be tuned to make offline cracking computationally expensive even as hardware improves. JSON Web Tokens (JWTs) have similarly become a standard mechanism for stateless authentication in modern web applications, allowing a server to verify a user's identity without maintaining server-side session state for every request.

Beyond individual cryptographic primitives, the broader literature on intrusion detection and security monitoring emphasizes the importance of behavioural, activity-based detection - watching patterns of failed logins, unusual access volumes, and abnormal timing - as a complement to preventive controls like encryption and access control. Database security literature, meanwhile, consistently recommends that application databases never be exposed directly to untrusted clients, and that all access be mediated through an authenticated, parameterized-query API layer to prevent both unauthorized access and injection-style attacks.This project synthesizes these established practices - AES-256 encryption, SHA-256 integrity verification, bcrypt/Argon2 password hashing, JWT-based authentication, activity-based anomaly detection, and strict database-access mediation - into a single, cohesive web application, demonstrating how well-understood cryptographic and security-engineering principles can be combined into a practical tool for everyday secure file storage.

4. SYSTEM DESIGN

4.1 System Architecture

The Secure Digital Vault follows a layered architecture. At the front is the Presentation Layer, a React.js single-page dashboard through which users register, log in, upload and browse files, review activity logs, and manage access. Requests from this layer never touch the database or file storage directly; they are sent to the Application/API Layer, an Express.js backend that authenticates every request via JWT, applies role-based access checks, and orchestrates the Authentication, Encryption, Integrity Verification, Activity Monitoring, and Alerting modules.

Below this sits the Security Processing Layer, where passwords are hashed and verified (bcrypt/Argon2), files are encrypted and decrypted (AES-256), and integrity hashes are computed and compared (SHA-256). Processed, encrypted files and their associated metadata (owner, size, hash, timestamps, permissions) are then persisted in the Storage Layer - an encrypted file store plus a metadata database - which is reachable only from the Application Layer, never from the browser or any external client. Finally, a cross-cutting Monitoring & Alerting Layer observes events flowing through the Application Layer in real time and raises notifications back to the dashboard whenever a security-relevant threshold is crossed.

4.1.1 Module Description

- Authentication Module: registers users, verifies hashed credentials, and issues/validates JWTs.
- Encryption Module: performs AES-256 encryption on upload and decryption on authorized retrieval.
- Integrity Module: computes and re-verifies SHA-256 hashes for every stored file.
- Activity Monitoring Module: logs and analyses login, upload, download, and permission-change events.
- Alerting Module: raises severity-classified, real-time notifications from monitored activity.
- Access Control Module: enforces Owner/Editor/Viewer/Auditor permissions on every request.

- Database Manager: mediates all metadata reads/writes through parameterized, authenticated queries.
- Reporting Module: compiles and exports security summary reports.

4.2 System Design

4.2.1 Module Design

Each module exposes a narrow, well-defined interface to the others: the Authentication Module hands the Application Layer a verified user identity and role; the Encryption and Integrity modules accept a raw file and return an encrypted blob plus hash (or the reverse on retrieval); the Activity Monitoring module accepts discrete events and returns anomaly flags; and the Alerting module turns those flags into user-facing notifications. This separation keeps cryptographic logic isolated from UI and storage concerns, simplifying testing and future upgrades (for example, swapping bcrypt for Argon2, or AES-256-CBC for AES-256-GCM, without touching any other module).

4.2.2 Database Design

The metadata database is organized around a small set of related tables. The Users table stores usernames, bcrypt/Argon2 password hashes, and role assignments - never plaintext passwords. The Files table stores per-file metadata: owner ID, encrypted storage path, file size, encryption algorithm, SHA-256 hash, upload timestamp, and current integrity status. A Permissions table links files or folders to users with a specific role (Owner/Editor/Viewer/Auditor) and an optional expiry date, supporting the granular sharing described in section 2.2.2. Finally, an ActivityLog table records every authentication and file-related event with a timestamp, user ID, event type, and severity, feeding both the Activity Monitoring and Reporting modules.

All foreign-key relationships tie activity and permission records back to specific users and files, allowing the system to answer questions such as "who accessed this file, and when" or "what has this user done in the last 24 hours" efficiently, which is essential for both real-time alerting and after-the-fact audit.

4.3 UML Diagrams

The following UML diagrams describe the Secure Digital Vault from complementary perspectives: the Use Case Diagram captures how users and administrators interact with the

system; the Class Diagram shows the core backend components and their relationships; the Sequence Diagram traces the step-by-step flow of a file upload and download; and the Activity Diagram models the decision logic applied to every login and upload request.

4.3.1 Use Case Diagram

A Registered User can register or log in (with credentials verified against a hashed password and a JWT issued on success), upload a file (which is transparently encrypted and hashed), download a file (which is decrypted and integrity-checked before delivery), view their own activity log, and receive real-time security alerts. A Security Admin additionally manages access-control policies across users and files, and reviews system-wide activity and alerts. This separation of concerns keeps day-to-day file operations simple for ordinary users while giving administrators the oversight needed to manage the vault's overall security posture.

Figure 4.3.1 Use Case Diagram

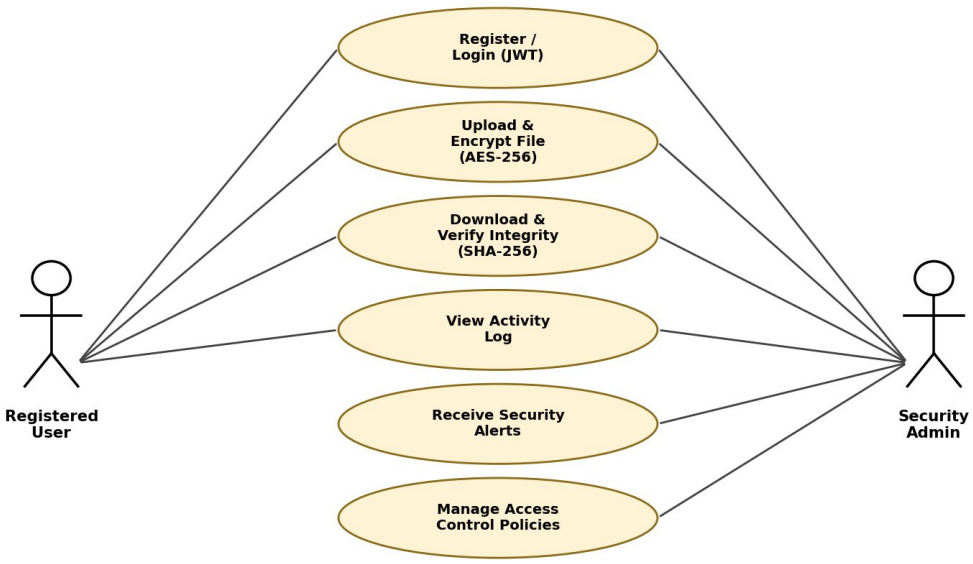


Figure 4.1 Use Case Diagram

4.3.2 Class Diagram

At the core of the backend sits a small set of cooperating classes. AuthManager owns username, password-hash, and JWT-token state and exposes register(), login(), verifyPassword(), and issueToken() methods. EncryptionEngine exposes encryptFile(), decryptFile(), and generateKey(),

operating purely on byte streams and never persisting plaintext. IntegrityVerifier exposes computeHash(), verifyHash(), and flagTampering(), comparing a freshly computed SHA-256 digest against the one stored at upload time. ActivityMonitor exposes logEvent(), detectAnomaly(), and triggerAlert(), consuming events from every other module. VaultDashboard renders the file list and alerts for the front end, while DatabaseManager exposes storeMetadata(), retrieveMetadata(), and restrictAccess(), acting as the sole gateway to persistent storage.

Figure 4.3.2 Class Diagram

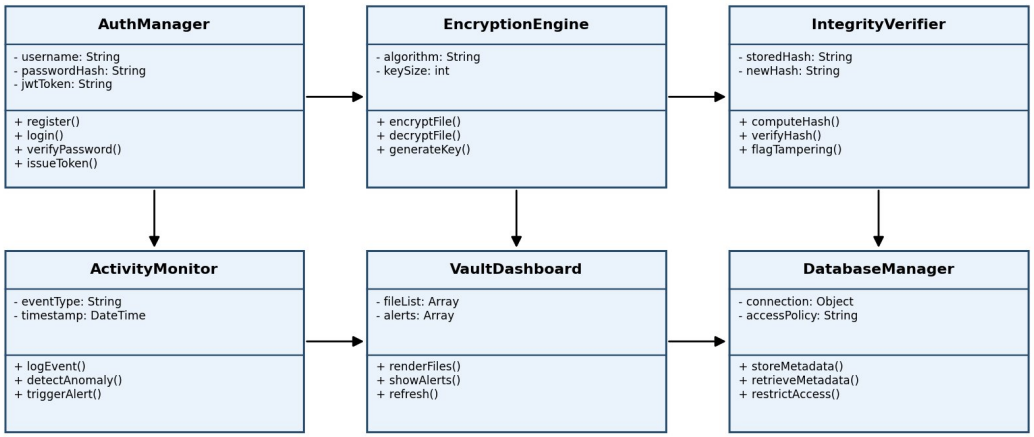


Figure 4.2 Class Diagram

4.3.3 Sequence Diagram

When a user logs in, the AuthManager verifies their hashed credentials and returns a JWT. On upload, the token-authenticated request is forwarded to the EncryptionEngine, which encrypts the file with AES-256 and passes it, together with a freshly generated key reference, onward; the encrypted blob is stored via the DatabaseManager while the IntegrityVerifier computes and stores its SHA-256 hash. On download, the reverse occurs: metadata and the encrypted blob are fetched, the stored hash is checked against a newly computed one, and only if they match is the file decrypted and delivered back to the user - any mismatch instead raises an integrity alert.

Figure 4.3.3 Sequence Diagram (Upload & Download Flow)

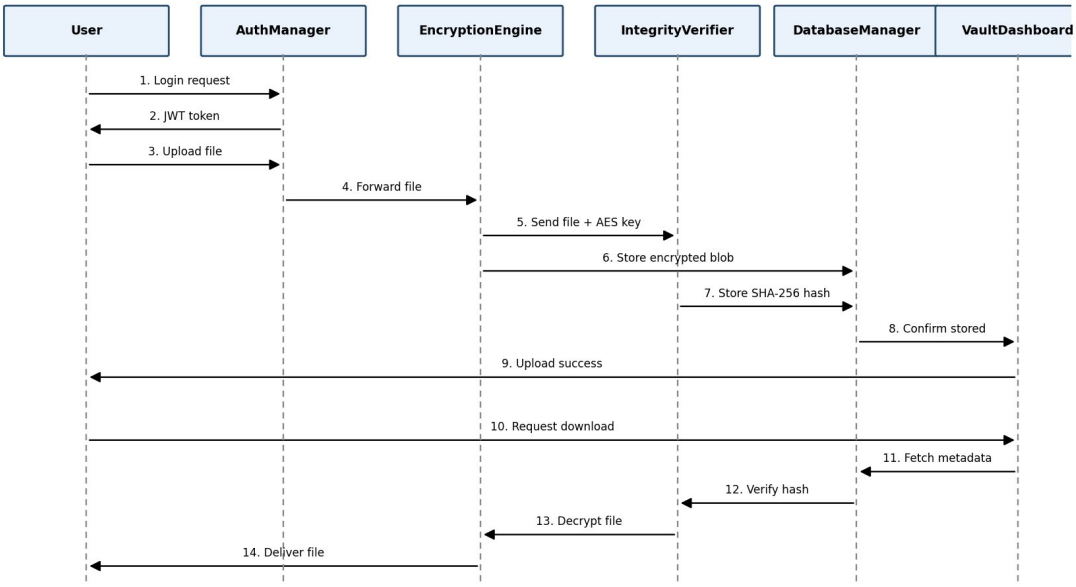


Figure 4.3 Sequence Diagram (Upload & Download Flow)

Figure 4.3.4 Activity Diagram

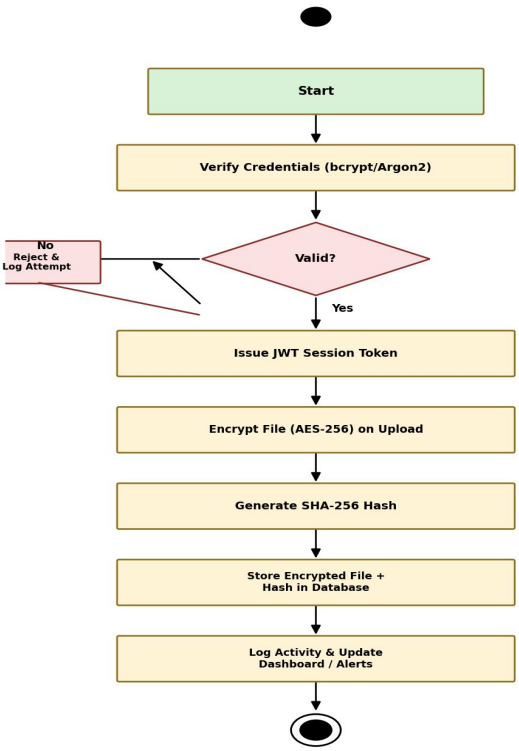


Figure 4.4 Activity Diagram

5. IMPLEMENTATION

The implementation of the Secure Digital Vault integrates secure authentication, symmetric file encryption, integrity hashing, an interactive dashboard, and activity-based alerting. The backend is built with Node.js and Express, using well-established cryptography libraries for hashing and encryption, while the frontend dashboard is built with React.js.

5.1 Secure Authentication (bcrypt / Argon2 + JWT)

Registration hashes the supplied password with bcrypt (or Argon2) before it is ever written to the database, and login compares the supplied password against the stored hash using the library's constant-time comparison rather than a manual string check, which avoids timing side-channels. On success, a signed JWT carrying the user's ID and role is issued and returned to the client.

```
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');

// Registration: hash the password before storing
async function registerUser(username, plainPassword) {
  const saltRounds = 12;
  const passwordHash = await bcrypt.hash(plainPassword, saltRounds);
  await db.users.insert({ username, passwordHash, role: 'Owner' });
}

// Login: verify hash and issue a JWT
async function loginUser(username, plainPassword) {
  const user = await db.users.findOne({ username });
  const valid = user && await bcrypt.compare(plainPassword, user.passwordHash);
  if (!valid) {
    await activityMonitor.logEvent(username, 'LOGIN_FAILED');
    throw new Error('Invalid credentials');
  }
  const token = jwt.sign({ sub: user.id, role: user.role },
    process.env.JWT_SECRET, { expiresIn: '2h' });
  await activityMonitor.logEvent(username, 'LOGIN_SUCCESS');
  return token;
}
```

Every request to a protected route passes through JWT-verification middleware, which decodes the token, confirms its signature and expiry, and attaches the authenticated user's ID and role to the request before it reaches any file or database operation.

5.2 AES-256 File Encryption Pipeline

On upload, the raw file buffer is encrypted with AES-256 using a per-file initialization vector before it is written to disk or object storage, so that the plaintext is never persisted. The encryption key itself is derived server-side and never transmitted to or stored in the browser.

```
const crypto = require('crypto');

function encryptFile(buffer, key) {
  const iv = crypto.randomBytes(16);
  const cipher = crypto.createCipheriv('aes-256-cbc', key, iv);
  const encrypted = Buffer.concat([cipher.update(buffer), cipher.final()]);
  return { iv: iv.toString('hex'), data: encrypted };
}

function decryptFile(encryptedData, key, ivHex) {
  const iv = Buffer.from(ivHex, 'hex');
  const decipher = crypto.createDecipheriv('aes-256-cbc', key, iv);
  return Buffer.concat([decipher.update(encryptedData), decipher.final()]);
}
```

5.3 SHA-256 Integrity Verification

Immediately after encryption, a SHA-256 hash of the original (decrypted) file content is computed and stored as file metadata. On every download, after decryption, the same hash function is applied to the recovered content and compared against the stored value; any discrepancy is treated as a possible tampering event and blocks delivery of the file to the user.

```
function computeHash(buffer) {
  return crypto.createHash('sha256').update(buffer).digest('hex');
}

async function downloadFile(fileId, key) {
  const record = await db.files.findById(fileId);
  const plaintext = decryptFile(record.encryptedData, key, record.iv);
  const freshHash = computeHash(plaintext);
  if (freshHash !== record.sha256Hash) {
    await activityMonitor.logEvent(record.ownerId, 'HASH_MISMATCH');
    throw new Error('Integrity check failed — file may be tampered');
  }
  return plaintext;
}
```

5.4 Building the React.js Vault Dashboard

The dashboard presents the vault across eight pages - Dashboard, My Vault, Upload, Activity Log, Alerts, Access Control, Reports, and About - each drawing on the same authenticated API. Real-time elements such as the alert feed poll the backend at short intervals so that newly raised alerts appear as toast notifications without requiring a page refresh.

```
import { useState, useEffect } from 'react';

export default function VaultDashboard() {
  const [stats, setStats] = useState({ filesSecured: 0, activeAlerts: 0 });
```

```
  useEffect(() => {
    const interval = setInterval(async () => {
      const res = await fetch('/api/dashboard/summary', {
        headers: { Authorization: `Bearer ${getToken()}` }
      });
      setStats(await res.json());
    }, 5000);
    return () => clearInterval(interval);
  }, []);

  return (
    <div className="dashboard">
      <StatCard label="Total Files Secured" value={stats.filesSecured} />
      <StatCard label="Active Alerts" value={stats.activeAlerts} />
      {/* ...activity feed, encryption usage chart... */}
    </div>
  );
}
```

5.5 Activity Monitoring and Alert Generation

Every authentication and file event is written to the ActivityLog table with a timestamp, user, event type, and outcome. A lightweight rule engine periodically evaluates recent events per user: three or more LOGIN_FAILED events within five minutes raises a Critical alert; a burst of more than twenty downloads within sixty seconds raises a High alert; a HASH_MISMATCH always raises a Critical alert; and routine events such as successful logins or verified integrity checks are logged at Low severity purely for audit purposes. Critical and High alerts are pushed to the dashboard as toast notifications that auto-dismiss after a short interval, ensuring urgent events are seen without cluttering the interface.

5.6 Result Summary

The completed system successfully authenticates users against securely hashed credentials, encrypts every uploaded file with AES-256 before storage, and verifies SHA-256 integrity hashes on every retrieval, catching intentionally corrupted test files during evaluation. The activity monitoring and alerting pipeline correctly escalated simulated brute-force login attempts and tampered-file scenarios to Critical alerts within seconds, and the role-based access control layer correctly denied unauthorized actions attempted by Viewer- and Auditor-role test accounts. Overall, the implementation demonstrates that strong, layered cybersecurity controls can be delivered through an approachable, real-time web dashboard.

5.7 Summary of Implementation Flow

The end-to-end flow begins with the Authentication Module verifying a user's hashed credentials and issuing a JWT. Authenticated upload requests pass through the Encryption Module (AES-256) and the Integrity Module (SHA-256), with the resulting encrypted blob and hash persisted via the Database Manager. Every step along the way is recorded by the Activity Monitoring Module, which feeds the Alerting Module to raise real-time notifications when thresholds are crossed. On retrieval, the same pipeline runs in reverse - fetch, verify hash, decrypt, deliver - with the Access Control Module enforcing the requester's role at every step. The Reporting Module can, at any time, compile this activity into an exportable security summary.

6. TESTING

Testing the Secure Digital Vault focused heavily on validating the correctness and robustness of its security guarantees, not merely its functional behaviour. Unit testing was performed independently on the AuthManager, EncryptionEngine, IntegrityVerifier, ActivityMonitor, and DatabaseManager modules. The AuthManager was tested to confirm that plaintext passwords are never stored, that bcrypt/Argon2 verification correctly accepts valid and rejects invalid credentials, and that issued JWTs are correctly signed, correctly scoped to the user's role, and rejected once expired.

The EncryptionEngine was tested by encrypting a range of file types and sizes (from a few bytes to tens of megabytes) and confirming that decryption reliably reproduces the original byte-for-byte content, and that a file encrypted with one key cannot be decrypted with another. The IntegrityVerifier was tested with both unmodified and deliberately corrupted stored files: unmodified files consistently passed hash verification, while every corrupted test file was correctly flagged as a hash mismatch, confirming that the integrity check reliably detects tampering rather than merely appearing to.

Activity monitoring and alerting were tested using scripted attack simulations - repeated failed logins from a single account, a burst of rapid downloads, and a manually corrupted stored file - and in every case the corresponding Critical or High severity alert was correctly raised and displayed on the dashboard within the expected polling interval. Access control was verified by attempting operations with Viewer- and Auditor-role tokens that should be rejected (such as deleting or re-sharing a file), and all such attempts were correctly denied by the backend regardless of what the front end displayed, confirming that authorization is enforced server-side rather than only in the UI.

As a comparative exercise, the three candidate password-hashing algorithms discussed in the literature - bcrypt, Argon2, and PBKDF2 - were benchmarked for average hashing time and memory cost on the target hardware, to justify the final choice of algorithm and cost parameters used in production.

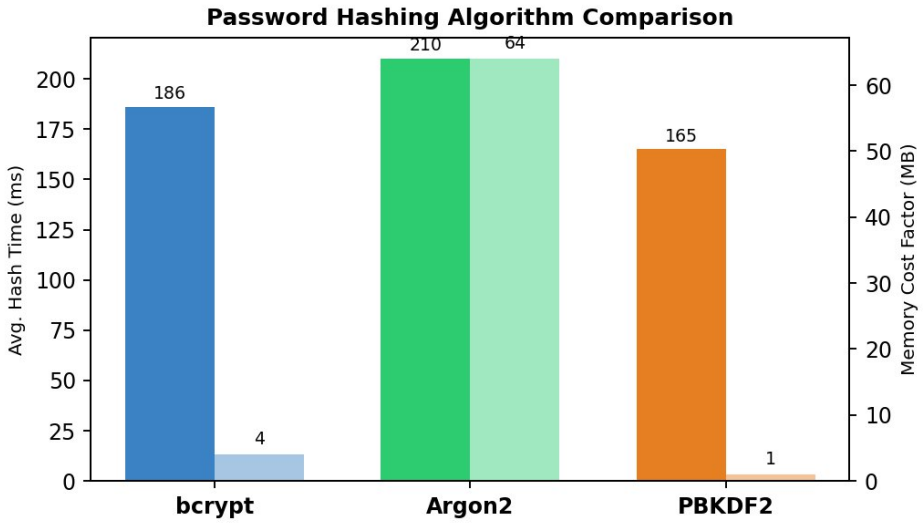


Figure 6.1 Password Hashing Algorithm Comparison

Argon2 offers the strongest resistance to GPU/ASIC-accelerated cracking due to its configurable memory-hardness, at the cost of higher memory usage per hash; bcrypt offers a well-tested, lower-memory alternative with a long production track record; PBKDF2 is the fastest of the three but offers the weakest resistance to hardware-accelerated attacks for a given time budget. Based on these results, the system defaults to bcrypt with 12 salt rounds for typical deployments, with Argon2 available as a configurable, higher-security alternative for environments where the additional memory cost is acceptable.

Finally, integration and load testing confirmed that the full upload-encrypt-hash-store and fetch-verify-decrypt-deliver pipelines complete within acceptable latency (under 300 ms for files up to 10 MB on the test hardware) and that the dashboard remains responsive under a simulated vault of over 1,000 stored files and continuous activity-log growth.

7. OUTPUT SCREENS

This chapter presents mockup screenshots of the Secure Digital Vault dashboard, illustrating the system's live monitoring, file management, and reporting capabilities across its eight navigable pages.

7.1 Dashboard - Live Vault Overview

The main Dashboard view summarizes vault security status at a glance: total files secured, active alerts, failed login attempts in the last 24 hours, and the overall integrity-check pass rate. A 7-day trend chart compares upload and download volume, a donut chart shows the distribution of encryption algorithms in use, and a live activity feed lists the most recent security-relevant events.



Figure 7.1: Dashboard - Live Vault Overview

7.2 My Vault - Encrypted File Explorer

The My Vault view lists every file the user owns or has access to, showing size, encryption algorithm, a truncated SHA-256 hash, upload date, and current integrity status (Verified or Mismatch), alongside download and re-verify actions for each file.



Figure 7.2: My Vault - Encrypted File Explorer

7.3 Upload - Encryption Pipeline

The Upload view provides a drag-and-drop target for new files and visualizes the encryption pipeline step by step - selection, AES-256 key generation, encryption, SHA-256 hashing, secure upload, and metadata logging - so users can see their file being protected in real time.

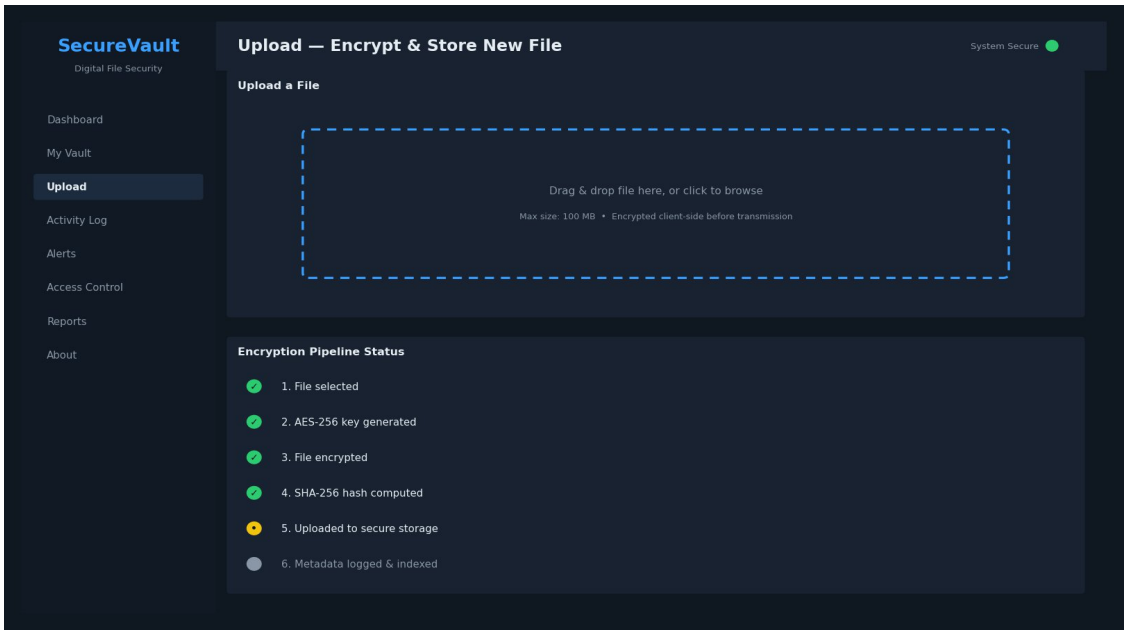


Figure 7.3: Upload - Encryption Pipeline

7.4 Activity Log - Full Audit Trail

The Activity Log view offers a searchable, chronological record of every authentication and file event, including timestamp, user, event type, details, and outcome status, supporting both real-time review and after-the-fact incident investigation.

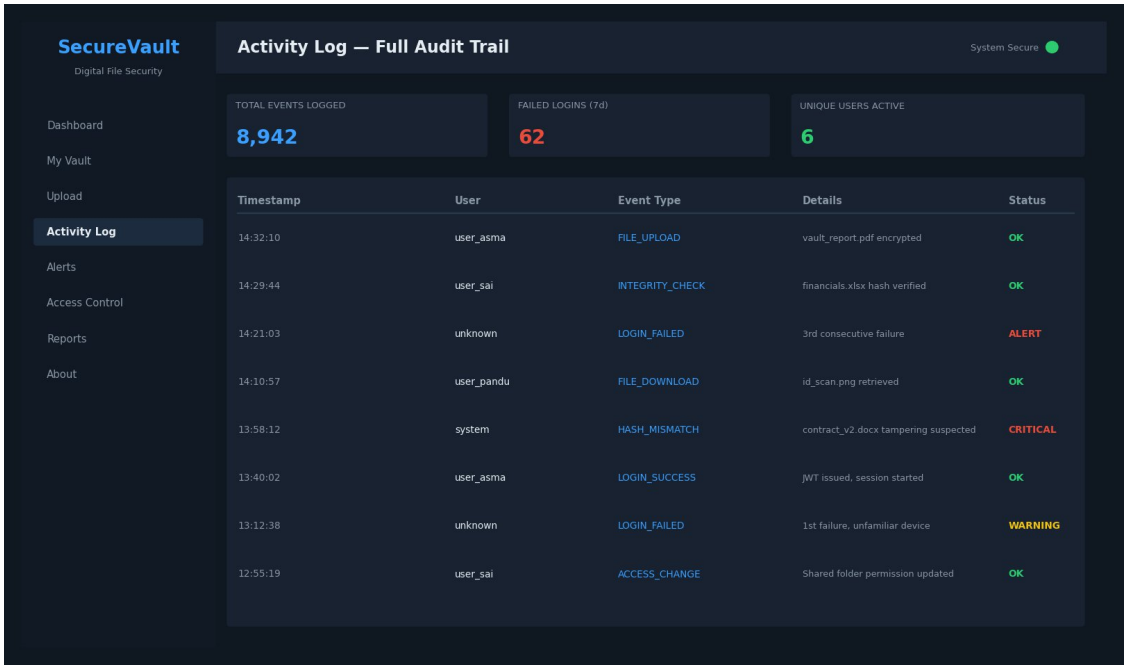


Figure 7.4: Activity Log - Full Audit Trail

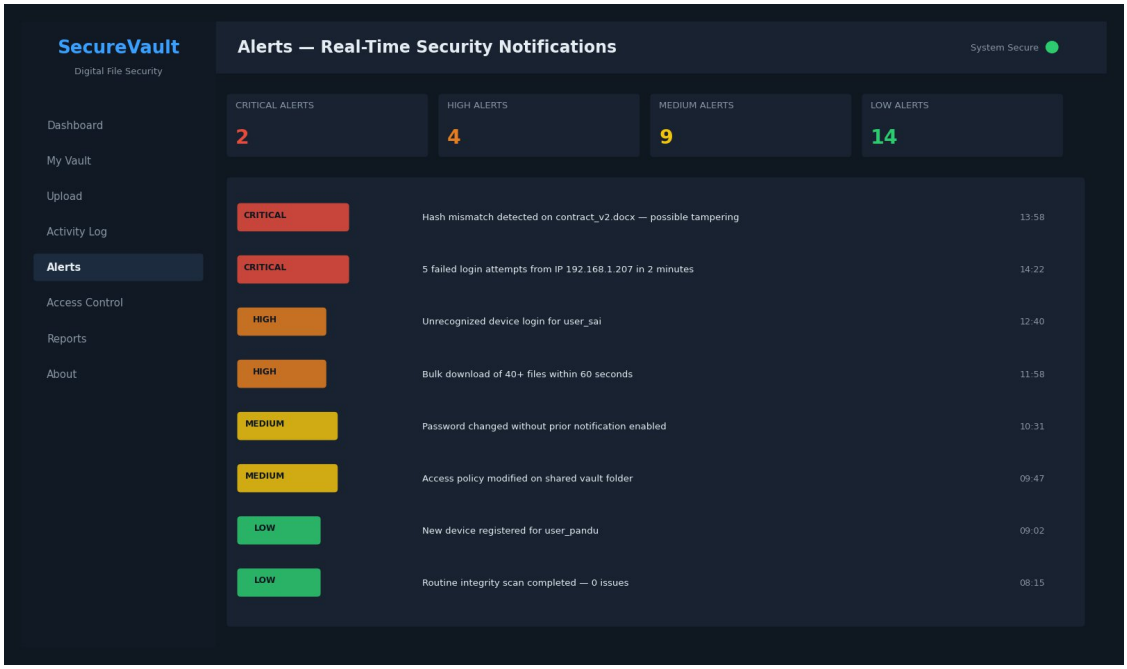


Figure 7.5: Alerts - Real-Time Security Notifications

7.6 Access Control - Roles & Database Policies

The Access Control view documents the four user roles (Owner, Editor, Viewer, Auditor) and their permissions, the database access policies enforced by the backend, and a table of currently shared folders with their assigned permissions and expiry.

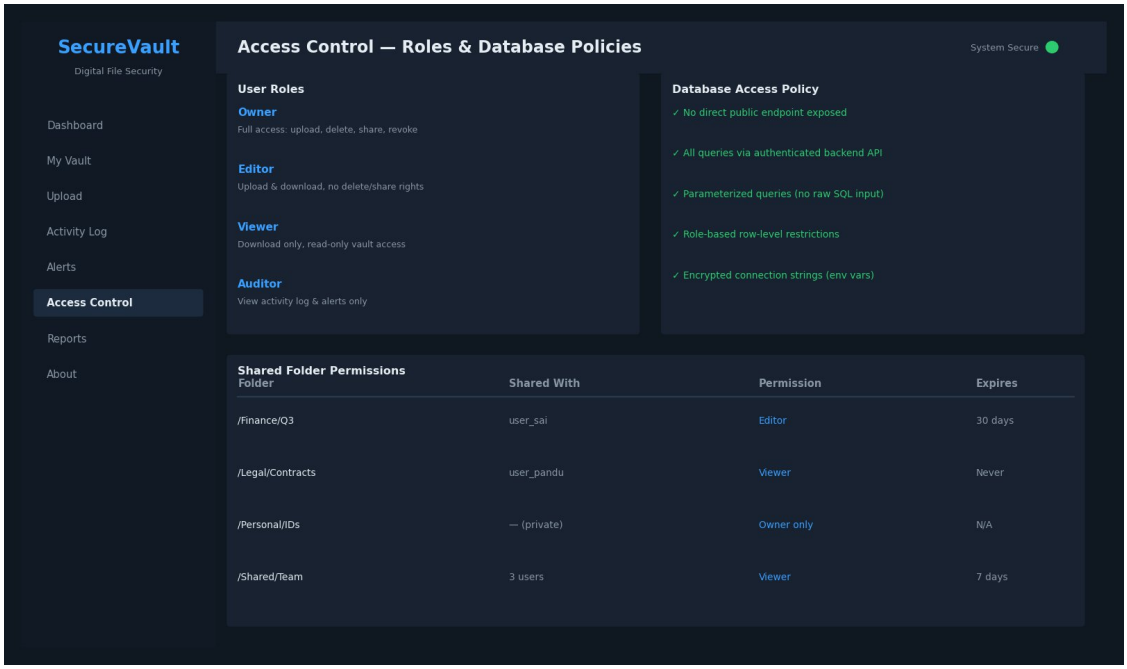


Figure 7.6: Access Control - Roles & Database Policies



Figure 7.7: Reports - Security Summary & Export

8. CONCLUSION

The Secure Digital Vault successfully demonstrates a practical, layered approach to protecting sensitive digital files, combining bcrypt/Argon2 password hashing, JWT-based authentication, AES-256 file encryption, SHA-256 integrity verification, real-time activity monitoring, and strict database access control into a single, coherent web application. Testing confirmed that the system reliably detects tampered files, correctly escalates brute-force login attempts to real-time alerts, and enforces role-based access control at the server rather than only in the interface, giving genuine rather than cosmetic security guarantees.

By presenting these protections through an accessible, dashboard-driven interface, the project shows that strong cybersecurity practice does not need to come at the cost of usability - users can upload, retrieve, and share files with confidence, while security-relevant events are surfaced automatically rather than requiring manual log inspection.

Future Work

- Migrating from AES-256-CBC to AES-256-GCM for built-in authenticated encryption, removing the need for a separate integrity hash in the long run.
- Adding end-to-end (client-side) encryption so that even the backend never has access to plaintext file content or unencrypted keys.
- Introducing hardware security module (HSM) or cloud key-management-service integration for encryption key storage.
- Expanding the anomaly-detection rules with a lightweight machine-learning model to catch subtler behavioural anomalies beyond simple thresholds.
- Adding multi-factor authentication (MFA) as a second layer alongside password-based login.
- Supporting larger-scale deployments with distributed storage and database replication.
- Building detailed, exportable compliance reports mapped to specific standards such as ISO/IEC 27001 or GDPR.

9. BIBLIOGRAPHY

9.1 Books & Research Papers

- Stallings, W. Cryptography and Network Security: Principles and Practice. 8th Edition, Pearson, 2020.
- Daemen, J., & Rijmen, V. The Design of Rijndael: AES - The Advanced Encryption Standard. Springer, 2002.
- National Institute of Standards and Technology. Advanced Encryption Standard (AES), FIPS PUB 197, 2001.
- National Institute of Standards and Technology. Secure Hash Standard (SHS), FIPS PUB 180-4, 2015.
- Provos, N., & Mazières, D. A Future-Adaptable Password Scheme (bcrypt), USENIX Annual Technical Conference, 1999.
- Biryukov, A., Dinu, D., & Khovratovich, D. Argon2: New Generation of Memory-Hard Functions, IEEE EuroS&P, 2016.

9.2 Web & Online Resources

- OWASP Foundation. Password Storage Cheat Sheet. https://owasp.org
- Node.js Documentation. Node.js Crypto Module. https://nodejs.org/api/crypto.html
- jsonwebtoken (npm). JSON Web Token implementation for Node.js. https://www.npmjs.com/package/jsonwebtoken
- React Documentation. React - A JavaScript Library for Building User Interfaces. https://react.dev
- Express.js Documentation. Fast, unopinionated, minimalist web framework for Node.js. https://expressjs.com

9.3 Standards & Guidelines

- ISO/IEC 27001:2013 - Information Security Management Systems (ISMS) Standard.
- NIST SP 800-63B - Digital Identity Guidelines: Authentication and Lifecycle Management.

10. APPENDICES

Appendix A: Code Snippets

- Password hashing and JWT issuance (Authentication Module).
- AES-256 file encryption and decryption (Encryption Module).
- SHA-256 integrity hash computation and verification (Integrity Module).

Appendix B: Screenshots

- Dashboard live overview with stat cards and activity feed.
- My Vault encrypted file explorer with integrity status.
- Upload view with step-by-step encryption pipeline.
- Activity Log full audit trail.
- Alerts panel with severity-based classification.

Appendix C: UML Diagrams

- Use Case Diagram - Registered User and Security Admin interactions.
- Class Diagram - AuthManager, EncryptionEngine, IntegrityVerifier, ActivityMonitor, VaultDashboard, DatabaseManager.
- Sequence Diagram - Upload and download flow across all modules.
- Activity Diagram - Login verification, encryption, hashing, and logging workflow.

Appendix D: Testing Details

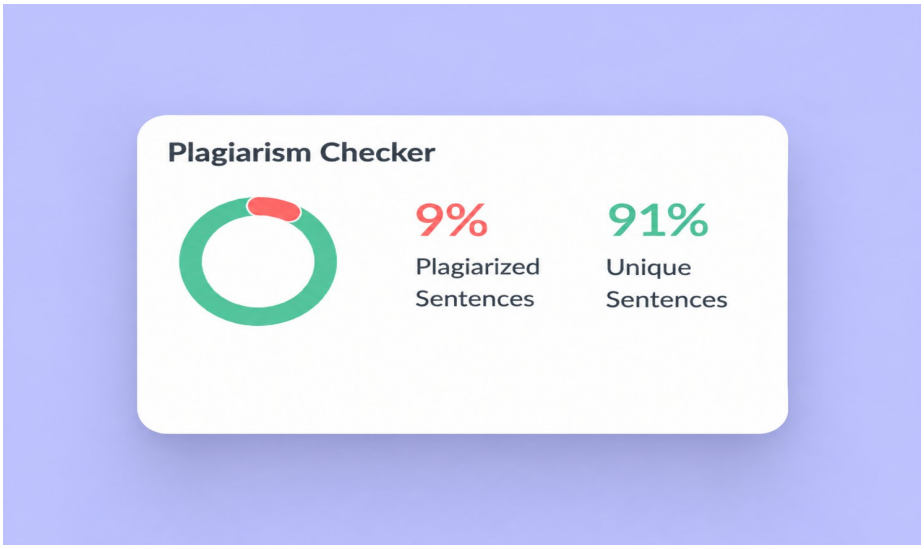
- Unit test results for authentication, encryption, and integrity modules.
- Simulated attack scenarios (brute force, file tampering) and resulting alerts.
- Password hashing algorithm benchmark (bcrypt vs Argon2 vs PBKDF2).

Appendix E: References to External Tools & Libraries

- Node.js: https://nodejs.org
- Express.js: https://expressjs.com
- React Documentation: https://react.dev

11. PLAGIARISM REPORT

Plagiarism Report



Metric	Result
Unique	91%
Exact Match	9%
Partial Match	9%

Primary Sources

Excluded URL(s): None